01 Executive Summary

MR Risk Assistant: AI-assisted merge request risk triage for code review TEAM018 | M Mahadev (2025mb26043) | MBA ZG583 Management of AI Products

---

Problem

In most engineering teams, every merge request (MR) goes through roughly the same review, no matter what it changes. A typo fix in a README and a change to login logic sit in the same queue and often get the same 10-minute look. Reviewers are busy, so in practice the queue is worked oldest-first or by whoever shouts loudest.

The result is that review effort is spread evenly while risk is not. The changes that actually hurt us (a leaked key, an unsafe Kubernetes setting, a payment calculation without tests, a migration that locks a table) usually look small and pass without anyone senior looking at them. Large but harmless MRs, like UI refactors or translation updates, get the extra attention instead.

Target user

- **Primary:** the reviewer / tech lead who has 15-30 open MRs and has to decide where to spend their limited time.
- **Secondary:** the engineering manager who decides who reviews what, and the security / platform team who should be pulled in for specific kinds of change.

Product concept

A review assistant that sits in the MR workflow. For each MR it:

1. reads the diff and metadata (paths, size, test changes, risky patterns),
2. gives a risk score and a Low / Medium / High band,
3. explains *why* in plain language,
4. suggests a review path (standard, senior, security, infra, add tests),
5. lets the reviewer accept or override, with a reason.

It does **not** approve or block merges. The human reviewer stays accountable.

Why AI?

Static rules (CODEOWNERS, "big MR = careful review") already exist and they help, but they miss what matters. In my sample set, the "look harder at MRs over 300 lines" rule caught only 1 of 11 problem MRs and 4 of its 5 flags were false alarms. Risk depends on a *combination* of signals (where the change is, whether tests changed, what patterns appear) and the right weights differ between codebases. That is a learning problem. Separately, an LLM is good at the part rules are bad at: reading a diff and writing a short, specific explanation a reviewer can act on.

So the design is a **hybrid**: deterministic scanners and rules for things that must never be missed (secrets, privileged containers), an ML risk score learned from our own MR history, and an LLM only for the explanation.

Recommendation

Prototype done, proceed to a narrow pilot. Do not scale yet.

Pilot with one team / 2-3 repositories for 8 weeks in "shadow then advisory" mode. Move forward only if the proof gates in the roadmap are met: it catches more problem MRs than the current process without flagging more than ~30% of the queue, and reviewers keep the override rate below ~25%. If reviewers ignore it or it floods them with false alarms, stop.

The prototype (prototype/index.html) shows the workflow end to end on 24 synthetic MRs. It proves the interaction and the trust design. It does not prove real-world accuracy, because I wrote both the rules and the test data.